

27. Strojové učení (učení s učitelem, učení bez učitele, posilované učení)

27. Strojové učení

SZZ okruh 27 (FIT BUT). Schopnost systému měnit znalosti, aby činnost vykonával efektivněji.

Shrnutí

Učení s učitelem

- Trénovací množina = vstupní vektory + **správné výstupy** (labels); data se dělí na **trénovací** (~80 %) / **testovací** / **validační**, aby model **generalizoval** na neznámá data.
- **Rozhodovací stromy** — klasifikace dle atributů; **ID3** vybírá atribut maximalizující **informační zisk** (minimalizující **entropii**).
- **Prohledávání prostoru verzí** (specific → general, candidate elimination); **klasifikace obrazů** (příznaková — diskriminační funkce, etalony; strukturální/syntaktická).

Učení bez učitele

- Hledání podobností a **shlukování** bez labelů; jediná vstupní informace je počet shluků.
- **k-means** — zařazení k nejbližšímu středu (Euklidova/Hammingova metrika), přepočtení středů průměrem, iterace do ustálení.

Posilované učení

- Učení z **odměn/penalizací** v koncových stavech a vlastních zkušenostech.
- **On-policy** (učí strategii, kterou hraje — SARSA) × **off-policy** (Q-learning — hraje náhodně, aktualizuje dle maxima).
- **TD-learning** (ohodnocuje stavy), **Q-learning** (ohodnocuje akce $Q(s,a)$).

Souvisí s [[okruhy/26-reseni-uloh-prohledavani|prohledáváním]] a se [[okruhy/31-pravdepodobnost-statistika|statistikou]].

[!note] Ke kontrole Zdroj poznamenává, že v novějších přednáškách je z učení s učitelem probírán hlavně rozhodovací strom + rozpoznávání obrazů a že prostor verzí / rozpoznávání se nemusí zkoušet — ověř rozsah u komise.

Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

Dělení ML □ [[#Učení s učitelem]] - *Tři typy a rozdíly?* □ S učitelem (známé labels), bez učitele (shlukování, bez labelů), posilované (odměna/penalizace).

Učení s učitelem □ [[#Učení s učitelem]] - *Rozhodovací stromy, ID3?* □ Klasifikace dle atributů; ID3 vybírá atribut s největším informačním ziskem. - *Entropie?* □ Míra neuspořádanosti množiny; 0 = vše v jedné třídě, max = rovnoměrné rozdělení. - *Proč dělit data na trénovací/testovací?* □ Aby se ověřila generalizace, ne jen zapamatování trénovací sady.

Učení bez učitele □ [[#Učení bez učitele]] - *k-means?* □ Náhodně k středů □ přiřazení nejbližšího □ přepočít středů průměrem □ iterace do ustálení.

Posilované učení □ [[#Posilované učení]] - *On-policy vs. off-policy?* □ On-policy (SARSA) učí hranou strategii; off-policy (Q-learning) aktualizuje dle nejlepší akce nezávisle na hraní.

Plné znění (ke studiu)

Materiál: Opora IZU, strana 104+

Pozn.: V přednáškách se vůbec Prohledávání prostoru neprobírá. V přednášce Strojové učení se u Učení s učitelem vyskytují pouze Rozhodovací stromy. V následující přednášce je potom probráno téma Rozpoznávání a klasifikace obrazů. Je tedy možné, že se Prohledávání prostoru zkoušet nebude. Dříve se probíralo všechno níže uvedené.

Strojové učení (ML, Machine Learning) je schopnost inteligentního systému **měnit své znalosti** tak, aby příště vykonával stejnou nebo podobnou činnost **účinněji a efektivněji**.

Učení s učitelem

Spočívá v tom, že pro každý krok učení je **známá požadovaná odezva** a systém je tak okamžitě informován o **aktuálním hodnocení jeho poslední akce**. Učení se provádí na tzv. **trénovací množině** příkladů **T**, kdy každý příklad je představován **množinou vstupních hodnot** (vstupním vektorem) a **množinou** správných/požadovaných **výstupních hodnot** (výstupním vektorem):

Často se **množina** dostupných dat **nepoužívá** k trénování **celá**, ale rozdělí se na dvě nebo tři podmnožiny. První (**trénovací, cca 80 % dat**) se použije k trénování, druhá (**testovací, cca 10 % až 20 % dat**) se použije k testování a třetí (**cca 10 % dat**) se někdy použije k doladění parametrů. [[media/szz-27/media/image11.png]]

Metody učení s učitelem pracují s vektory, které nabývají symbolických hodnot (případné **číselné hodnoty** jsou rovněž chápány jako symbolické) a jsou založené na předpokladu, že každá hypotéza, která **vyhovuje dostatečně velké množině trénovacích příkladů**, bude vyhovovat i dalším, dosud **neznámým příkladům**.

Tvorba rozhodovacích stromů (Decision Tree Building)

Rozhodovací stromy slouží ke klasifikaci objektů na základě hodnot jejich atributů/vlastností. Jsou vytvářeny ze známé množiny příkladů. Používají se při dolování dat z databází.

Příklad viz str 105: opora_izu_2024.pdf

Algoritmus Decision Tree

[[media/szz-27/media/image10.png]]

Jedná se o základní algoritmus pro budování **rozhodovacích stromů**. Má dva vstupní parametry:

- **množinu příkladů MP**: jednotlivé řádky tabulky viz výše,
- **množinu podmínkových atributů MA**.

Algoritmus funguje následovně:

1. Patří-li **všechny prvky** množiny příkladů **MP** do **stejně třídy**, vraťte **listový** uzel označený touto třídou, jinak pokračujte.
2. Je-li množina atributů **MA** prázdná, vraťte **listový** uzel označený **disjunkcí** všech tříd, do kterých patří prvky v množině příkladů **MP**, jinak pokračujte.
3. Vyberte atribut **Ai**, odstraňte jej z množiny atributů **MA** a učiňte jej kořenem **aktuálního** stromu. Nechť **MA-i** je množina atributů **MA** bez atributu **Ai**.
4. Pro každou hodnotu **Hji** vybraného atributu **Ai**:
 1. Vytvořte novou větev stromu označenou hodnotou **Hji**.
 2. Volejte rekurzivně algoritmus s parametry **MPji** a **MA-i**, kde množina **MPji** je podmnožinou všech prvků množiny příkladů **MP**, které **mají** hodnotu **Hji** atributu **Ai**.
 3. Připojte vrácený podstrom/uzel k této větví.

Jinak řečeno, vytváří **strom na základě rozhodovacích parametrů**. Hloubka stromu závisí na tom, jestli je možné nějaký rozhodovací atribut ignorovat, protože nehledě na jeho hodnotě jsou výsledky z trénovací množiny stejné.

Na **základě trénovací množiny**, viz tabulka výše, lze říct, že pokud má uživatel **adekvátní ručení** a **vysoký dluh** je **risk** poskytnutí úvěru **nízký** bez zohlednění jeho **příjmu** a **historie úvěru** (příjem a historie úvěru budou mít v realitě vliv při rozhodnutí o riziku, ale trénovací množina je neobsahuje, takže se podle nich nemůže naučit a musí je ignorovat). [[media/szz-27/media/image3.png]]

Algoritmus je jednoduchý, ale obsahuje zásadní problém, kterým je výběr atributu **Ai** v bodě 3. **Nevhodné** výběry vedou k **hlubokým** a **neefektivním** vyhledávacím stromům, přestože **optimální strom** může být poměrně **jednoduchý**.

Algoritmus ID3 - Iterative Dichotomiser 3 (Induction of Decision Tree)

Jedná se o modifikaci algoritmu Decision Tree, který **řeší** problém při **výběru rozhodovacího atributu Ai** v bodě 3. Výběr atributu není náhodný, ale je prováděn tak, aby byl **maximalizován informační zisk**, tj. aby byl **nejdříve** vybrán takový **atribut**, který **co nejvíce ovlivní výsledné rozhodnutí** (atribut, který ovlivňuje rozhodnutí úplně nejvíce je pak kořenem rozhodovacího stromu). Lze na to nahlížet také tak, že **množiny**, které vzniknou **rozdělením dle nějakého atributu** mají **co nejmenší míru entropie** (míru neuspořádanosti) tj. je v nich **co nejvíce prvků** spadajících pod stejný výsledek. Pokud jsou v množině 3 prvky, které spadají **všechny do jedné kategorie** výsledků (např. nízký risk), je **míra entropie** této množiny **nulová**. Pokud zde budou naopak 3 prvky spadající do **3 různých kategorií** (nízký, přiměřený a vysoký risk), je **míra entropie** této množiny **nejvyšší**. Vybíráme tedy takový **atribut**, který rozdělí množinu na podmnožiny s **nejmenší entropií** a přinese tak **největší informační zisk**.

Informační zisk s respektováním hodnot **atributu Příjem** (<15, 15-35, >35) se vypočítá takto: **MP1** (<15) = {1,4,7,11}, **MP2** (15-35) = {2,3,12,14}, **MP3** (>35) = {5,6,8,9,10,13} (už jen podle rozdělení prvků do množin je zřejmé, že MP1 bude mít nulovou entropii, MP2 bude mít z této trojice největší entropii a entropie MP3 bude mezi). [[media/szz-27/media/image12.png]]

[[media/szz-27/media/image24.png]]

Celková entropie rozdělení podle atributu Příjem je pak spočítána jako **vážený průměr** (váha je dána počtem prvků v množině) entropií podmnožin. [[media/szz-27/media/image14.png]]

Výsledný prohledávací strom vytvořený pomocí ID3 bude vypadat následovně:

Prohledávání prostoru verzí (Version Space Search)

[[media/szz-27/media/image25.png]]

Prohledávání prostoru verzí představuje soubor metod učení významných pojmů/hypotéz na základě **pozitivních** a **negativních** příkladů. Při učení se hledá takový **popis daného pojmu/objektu/hypotézy**, který **zahrnuje všechny pozitivní příklady** a **vylučuje všechny negativní příklady** z trénovací množiny příkladů. Trénovací množina **vždy** musí obsahovat **pozitivní** i **negativní** příklady. Příklad trénovací množiny pro identifikaci pojmu míč může být následující:

Algoritmus Specific to general search

[[media/szz-27/media/image8.png]]

Metoda učení prohledáváním prostoru, která pracuje **od specifického k obecnějšímu**.

1. Vytvořte dvě prázdné množiny **S** (Specific) a **N** (Negative).
2. Uložte do množiny **S** první **kladný příklad**. Pro každý další příklad **p** z trénovací množiny:
 1. je-li **p kladným příkladem**, pak pro každý pojem **s** $\in$ **S**
 1. jestliže pojem **s** nelze unifikovat (**s** je obecnější než **p** a **p** spadá do množiny prvků, které lze pomocí **s** sestavit) s příkladem **p**, pak jej nahraďte jeho **nejvíce specifickým zobecněním** (aby už vyjádřit pomocí **s** šel), které lze unifikovat s příkladem **p**,
 2. odstraňte z **S** všechny pojmy **s**, které jsou **více obecné** než jiné pojmy v **S**,
 3. odstraňte z **S** všechny pojmy **s**, které lze **unifikovat** s některým pojmem v **N**.
 2. je-li **p záporným příkladem**, pak
 1. odstraňte z **S** všechny pojmy **s**, které lze unifikovat (lze pomocí nich vyjádřit záporný prvek **p**) s příkladem **p**,
 2. přidejte příklad **p** do **N**.

Jinak řečeno s **pozitivními příklady** se výsledné řešení stává **obecnější** - přidávají se **možnosti správných řešení**. Negativní příklady naopak množinu možných řešení redukují (odstraňují ta řešení, která by správně identifikovala i špatný objekt). Průběh algoritmu na příkladu s míčem vypadá následovně, výsledkem je, že míč je **objekt(X,Y,koule)**: [[media/szz-27/media/image29.png]]

Algoritmus General to Specific Search

Metoda učení prohledáváním prostoru, která pracuje **od obecného ke specifickému**.

1. Vytvořte dvě prázdné množiny **G** (General) a **P** (Positive) a uložte do **G** **nejobecnější** pojem (všechny parametry jsou vyjádřené proměnnou).
2. Pro každý další příklad **p** z trénovací množiny:
 1. je-li **p záporným příkladem**, pak pro každý pojem **g** $\in$ **G**:
 1. jestliže pojem **g** lze unifikovat s příkladem **p**, pak jej nahraďte jeho **nejobecnější specializací**, kterou **nelze unifikovat** s příkladem **p**,
 2. odstraňte z **G** všechny pojmy **g**, které jsou **více specializované** než jiné pojmy v **G**,
 3. odstraňte z **G** všechny pojmy **g**, které nelze unifikovat s některým pojmem v **P**.
 2. je-li **p kladným příkladem**, pak:
 1. odstraňte z **G** všechny pojmy **g**, které nelze unifikovat s příkladem **p**,
 2. přidejte příklad **p** do **P**.

Jinak řečeno se **zápornými příklady** se z obecného řešení **stává konkrétnější**, protože je řešení **konkretizováno**, aby nevyhovovalo **záporným** řešením. **Kladné příklady** pak **odstraňují** ta řešení, pomocí kterých je **není možné vyjádřit** (unifikovat). Průběh algoritmu je na obrázku, výsledek je **objekt(X,Y,koule)**:

[[media/szz-27/media/image5.png]]

Candidate eliminations

Spojuje postupy předchozích algoritmů.

Vytvořte dvě prázdné množiny **G** (General) a **S** (Specific) a uložte do **G** nejobecnější pojem.

Uložte do množiny **S** první **kladný příklad**.

Pro každý další příklad **p** z trénovací množiny:

- je-li **p** **kladným příkladem** pak:
 - odstraňte z **G** všechny pojmy **g** $\in$ **G**, které nelze unifikovat s příkladem **p**,
 - pro každý pojem **s** $\in$ **S**:
 - * jestliže pojem **s** **nelze unifikovat** s příkladem **p**, pak jej nahraďte jeho **nejvíce specifickým zobecněním**, které lze unifikovat s příkladem **p**,
 - * odstraňte z **S** všechny pojmy **s**, které jsou **více obecné** než jiné pojmy v **S**,
 - * odstraňte z **S** všechny pojmy **s**, které nejsou **více specifické**, než některé pojmy v **G**.
- je-li **p** **záporným příkladem**:
 - odstraňte z **S** všechny pojmy **s**, které lze unifikovat s příkladem **p**,
 - pro každý pojem **g** $\in$ **G**:
 - * jestliže pojem **g** lze unifikovat s příkladem **p**, pak jej nahraďte jeho **nejvíce zobecněnou specializací**, kterou **nelze unifikovat** s příkladem **p**,
 - * odstraňte z **G** všechny pojmy **g**, které jsou **více specifické** než jiné pojmy v **G**,
 - * odstraňte z **G** všechny pojmy **g**, které nejsou více obecné než některé pojmy v **S**.

Jestliže **G = S** a obě množiny přitom **obsahují jediný pojem**, pak je výsledkem učení právě tento pojem. Příklad tohoto algoritmu, výsledek je **objekt(X,Y,koule)**: [[media/szz-27/media/image15.png]]

Význam a vztah množin **S** a **G** je na obrázku níže. Každý pojem, který **by byl obecnější** než nějaký pojem v **G**, **by zahrnoval některé negativní příklady**, každý pojem, který **by byl specifitější** než nějaký pojem v **S**, **by vylučoval některé pozitivní příklady**. Výsek s otazníky značí objekty, které nejsou v trénovací množině, ale na základě výsledků algoritmů **prohledávání prostoru verzí** je poté lze identifikovat, např.: **míč je kulatý objekt, na jehož barvě, ani velikosti nezáleží**.

Rozpoznávání a klasifikace obrazů (Pattern Recognition and Classification)

[[media/szz-27/media/image17.png]]

(Od roku 2021 se nikdo na rozpoznávání neptal. A dokonce i otázka zní jinak. Rozpoznávání je v 9. přednášce. Strojové učení (s učitelem / bez učitele / posilované učení) je 8. přednáška. Možná tedy spíše pro doplnění, nebo když máte v komisi jen zkoušející s IZU otázkami apod.)

Pro rozpoznávání obrazů existují různé algoritmy, které pracují s jedním z následujících popisů:

- **Příznakový** (popis vektory číselných příznaků): využívá **statických informací** o **příznacích** obrazů obsažených v množině trénovacích dat. Jde o tzv. **statické příznakové rozpoznávání**.

- **Strukturální/syntaktický** (popis strukturálními prvky, tzv. primitivy): využívá **vztahy mezi příznaky** obrazů rozpoznávaných objektů.

Zásadním předpokladem úspěšného rozpoznávání je výběr relevantních příznaků, resp. primitiv. [[media/szz-27/media/image16.png]]

Příznakové rozpoznávání

U příznakového rozpoznávání pracujeme s:

- **n-rozměrnými číselnými vektory** příznaků, které popisují rozpoznávané objekty,
- **množinou tříd**, do kterých rozpoznávané objekty chceme zařadit,
- **trénovací množinu**, která je tvořena **dvojicemi** skládajícími se z **n-rozměrného číselného vektoru a třídou**, do které vektor spadá.

Cílem je poté zařadit **libovolný** vektor příznaků do **jedné z tříd - klasifikovat** jej.

Metody **příznakového rozpoznávání/klasifikace** vycházejí z předpokladu, že obrazy objektů nebo jevů **stejných tříd** tvoří v **n-rozměrném** obrazovém prostoru **shluky**. Tyto shluky mohou být od sebe **zřetelně oddělitelné (separable)**, nebo se mohou **prolínat** a pak jsou **neoddělitelné (inseparable)**. Systémy, které se na trénovací množině naučí obrazy rozpoznávat a poté **klasifikují nové** obrazy, se nazývají **klasifikátory**.

Dichotomie Jedná se o **klasifikaci do dvou tříd**. V praxi je **dichotomie** poměrně **častá**, obecně jde klasifikovat do **n** tříd, což spočívá v nalezení **křivek/ploch/k-rozměrných útvarů**, které obrazy jednotlivých tříd od sebe **oddělují**. Toho se zajišťuje pomocí **diskriminačních funkcí** (pro každou třídu jedna), které obraz ohodnocují. **Obraz** je pak **umístěn/klasifikován** do třídy, jejíž **diskriminační funkce** má **největší** hodnotu (je nutné pro každý obraz vypočítat hodnoty všech diskriminačních funkcí). U dichotomie stačí jedna diskriminační funkce (respektive **2**, které od sebe **odečítáme**) a obrazy objektů klasifikujeme na základě **znaménka** do dvou tříd (kladná a záporná).

Algoritmus určení **lineárního klasifikátoru** pro dichotomii (diskriminační funkce):

1. Vynulujte vektor vah.
2. Nastavte indikátor změny **modif = false**.
3. Pro každý **vektor** z trénovací množiny, který **není správně klasifikován** (diskriminační funkce vrací jiné znaménko, než je v trénovacím příkladě) **upravte vektor vah** (kterým je vektor klasifikován) a nastavte **modif = true**.
4. Pokud došlo k úpravě vektoru vah (**modif = true**), tak se vraťte na bod 2.

Výsledkem bude rozdělení prostoru dle následujícího obrázku pro 2D.

Jeden klasifikátor klasifikující do **R tříd** může být nahrazen **R klasifikátory** pro dichotomii **naučených** na klasifikaci **patří/nepatří** do třídy **r**, $r \in \{1, R\}$. [[media/szz-27/media/image23.png]]

Etalony Jedná se o **těžiště shluků obrazů** jednotlivých tříd. Obraz je klasifikován do třídy, jejímuž **etalonu má nejbližší** (nejmenší vzdálenost v prostoru). Tato klasifikace však opět pracuje na principu klasifikace podle diskriminačních funkcí.

Rozpoznávání obrazů reprezentovaných neoddělitelnými třídami obrazů V případě **neoddělitelných tříd** obrazů již nelze rozhodnout, že **obraz x** patří do třídy **r**, ale lze konstatovat, že obraz **x** patří do třídy **r** s pravděpodobností **P(r | x)**. Úkolem trénovacích algoritmů je nalézt takový klasifikátor, který bude klasifikovat obrazy s co největší pravděpodobností, respektive minimalizovat ztrátu, která vzniká klasifikací do nesprávné třídy. Využívá se k tomu ztrátová matice, ztráta pro

správnou klasifikaci obrazu je **0** a pro **chybnou 1**. Na základě ztrát se matice zjednoduší na obrázek vpravo. [[media/szz-27/media/image20.png]]

[[media/szz-27/media/image13.png]]

Strukturální rozpoznávání

Jedná se o rozpoznávání obrazů na základě jeho popisu pomocí **primitiv**. Existují dva základní přístupy ke strukturálnímu rozpoznávání obrazů:

- rozpoznávání obrazů pomocí **gramatik - syntaktické rozpoznávání** (syntactic recognition)
- rozpoznávání obrazů **porovnáním se vzory** uloženými v databázi (template matching) - **neučili jsme se**.

Syntaktické rozpoznávání klasifikuje obrazy do **R** tříd pomocí **syntaktické analýzy**. Obrazy, tj. **řetězcové popisy objektů** nebo jevů, jsou přitom chápány jako **slova** a množina **primitiv** jako množina **terminálních symbolů**. Pokud pak gramatika generuje **všechna slova/obrazy**, která reprezentují obrazy třídy **r**, a negeneruje **žádné slovo**, která reprezentuje obraz **jiné třídy** (jinak řečeno všechny gramatiky generují vzájemně **různá slova**), lze klasifikaci převést na problém určení gramatiky, která jako jediná ze všech **R** gramatik generuje rozpoznávané slovo/obraz.

Na **úspěšnost** strukturálního/syntaktického rozpoznávání má velký vliv **výběr primitiv** (malá množina primitiv má malou vyjadřovací sílu, velká množina může být neovladatelná při učení).

Freemanův řetězcový kód Používá se ke strukturálnímu popisu obrysu objektu (obrazu). Za primitiva popisující obrys objektu považujeme **směry sousedů**. **Problémy** při popisu objektů tímto způsobem je jejich **natočení** a **startovací pozice** popisu (popis je invariantní vůči posuvu), což je činí prakticky nepoužitelným.

Problémy lze eliminovat následovně: [[media/szz-27/media/image18.png]]

- **natočení**: řešením je popis pomocí **relativního (diferenciálního) kódu**, který místo primitiv používá **rozdílů/diferencí natočení** mezi dvěma sousedními primitivy (následující – aktuální). Pro určení těchto rozdílů se postupuje v **opačném směru** (tj. po směru hodinových ručiček, overflow $2-3 = 3$, $1-3 = 2$, ... pokud je **první číslo menší**, je nutné k výsledku **přičíst 4**). Na obrázku lze po provedení postupu vidět, že **diferenční kódy** pro s_1 a s_{10} jsou stejné jako jsou stejné diferenční kódy s_2 a s_{20} .
- **startovací pozice**: řešení spočívá v tom, že řetězec **relativního (diferenciálního) kódu** rotujeme tak, aby řetězec číslic po převodu na číslo bylo **číslo největší** (tj. zleva obsahovalo největší číslice). Tomuto číslu se říká **číslo tvaru - shape number**. [[media/szz-27/media/image19.png]]

[[media/szz-27/media/image22.png]]

Číslo tvaru (shape number) je **invariantní vůči posunutí, natočení** objektu (pro 4 sousedy pouze **po 90°**) a **startovacímu bodu** popisu, je však **závislé na velikosti objektu**.

Freemanův řetězcový kód existuje i pro 8 sousedů, viz obrázek: [[media/szz-27/media/image7.png]]

Číslo tvaru [3 1 1 3 1 0 1 1 3 0 1 1] popisující objekt výše by generovala například **gramatika** s těmito prepisovacími pravidly:

- $S \rightarrow 3N$,
- $N \rightarrow 0N$,
- $N \rightarrow 1N$,
- $N \rightarrow 3N$,
- $N \rightarrow 1$

Tato gramatika by však **generovala i další** řetězce, které by daný objekt **nepopisovaly**, a proto k rozpoznávání by byla zřejmě **nepoužitelná**. Nalezení **relevantních** přepisovacích pravidel (tj. určení správné gramatiky) je mnohem **náročnější** než učení příznakových klasifikátorů nebo trénování neuronových sítí a dnes se tak moc nepoužívá.

Učení bez učitele

Učení bez učitele spočívá v **hledání podobnosti** mezi příklady **trénovací množiny** a v zařazování příkladů s **podobnými charakteristikami do skupin**. Umělý systém přitom **nedostává žádnou informaci** o správnosti klasifikace a jedinou informací, kterou má, resp. může mít je **počet skupin**, do kterých má příklady z trénovací množiny zařazovat/klasifikovat.

Příklady trénovací množiny jsou téměř vždy představované **číselnými vektory** příznaků klasifikovaných objektů či dějů. Metody **učení bez učitele** jsou pak založeny na předpokladu, že tyto vektory, resp. body, specifikují v příslušném **n-rozměrném** obrazovém prostoru **shluky**. Tyto **shluky** mohou představovat velmi **rozmanité n-rozměrné útvary**, například v několika souřadnicích může jít o zcela kompaktní shluky, zatímco v jiných souřadnicích mohou být značně rozptýlené.

k-means clustering

Algoritmus **klasifikuje** příklady (číselné vektory) z trénovací množiny do předem daného počtu shluků **k**. Algoritmus **zařazuje** vstupní vektor do toho **shluku**, k jehož **středu/těžišti má nejkratší vzdálenost**. Vstupem algoritmu je počet shluků **k** (musí být menší než počet vektorů) a průběh učení je následující:

1. Náhodně určí **k** rozdílných vektorů (často je vybere z trénovací množiny), které považuje za **středě (těžiště)** shluků.
2. Zařadí všechny vektory trénovací množiny do **příslušných shluků**, dle použité **metriky**: Nejčastěji se používá asi **Euklidovská** vzdálenost - délka úsečky mezi body, ale lze použít např. **Hammingovu** vzdálenost - je nejmenší počet pozic, na kterých se řetězce stejné délky daného kódu liší.
3. Zprůměrováním všech bodů ve shluku **přepočítá středy** ve všech shlucích.
4. Pokud se pozice ani jednoho středu nezměnila (tj. vektory byly opakovaně zařazeny do stejných shluků), algoritmus končí. Jinak pokračuje bodem 2.

Posilované učení

[[media/szz-27/media/image9.png]]

[[media/szz-27/media/image4.png]]

Posilované učení se od učení s učitelem odlišuje tím, že systém **ohodnocuje** své akce na základě **penalizací** či **odměn** získaných **v koncových stavech** a na základě **svých hodnocení** stavů/akcí získaných **vlastními** předchozími **zkušenostmi**.

On-Policy/Off-Policy

On-policy: On-Policy používá pro trénování i rozhodování stejná pravidla. Během učení vylepšuje současnou strategii tak, že využívá data získané právě následováním dané strategie. Např. pokud SARSA je ve stavu *s* a při prozkoumávání stavového prostoru volí další stav *s'* do kterého vstoupí stochasticky (náhodně), tak - řekněme že stav *s'* obsahuje 4 přechody - tak novou hodnotu stavu *s* bude počítat na základě náhodně vybraného přechodu stavu *s'*.

Off policy: Off-Policy používá pro finální rozhodování jiná pravidla než pro trénování. Během učení agent využívá data z různých zdrojů a strategií, které nemusí být shodné s tou, kterou agent prochází stavový prostor. Např. Q Learning prochází stavový prostor náhodně, ale při aktualizaci hodnoty stavu nevyužívá náhodné rozhodování, ale bere nejlepší přechod.

To pak vede k rozdílu cest, které jsou podle jednotlivých algoritmů ideální, jak je vidět zde:

[[media/szz-27/media/image28.png]]

Zdroj obrázku: <https://www.baeldung.com/cs/q-learning-vs-sarsa>

Policy-only learning

Princip tohoto algoritmu je založený na tom, že z **každého uzlu** grafu vedou **maximálně 2 cesty** (hrany), případně pro koncovou cestu pouze jedna cesta. každý uzel obsahuje **schránku s kameny** (analogie pravděpodobnosti) dvojí barvy - **bílé** a **černé** (pokud je počet bílých kamenů větší, je pravděpodobnější výběr cesty vlevo, naopak je to u většího počtu černých kamenů). [[media/szz-27/media/image27.png]]

Na začátku učení obsahuje schránka každého rozcestníku **stejný a dostatečný počet** černých a bílých kamenů. Učení pak probíhá na tomto principu, že se provádí náhodné procházky (náhodnost je dána pravděpodobností výběru cesty v uzlech). Pokud výsledná cesta:

- **skončí v cílovém** stavu (na obrázku uzel H), je do schránek na této cestě **přidán** kámen odpovídající barvy podle výběru směru - **odměna**. **Pravděpodobnost** výběru této cesty **se zvyšuje**.
- **skončí mimo cílový** stav (na obrázku uzly G, I, J), je ze schránek **odebrán** kámen odpovídající barvy podle výběru směru - **penalizace**. **Pravděpodobnost** výběru této cesty **se snižuje**.
[[media/szz-27/media/image26.png]]

Po ukončení učení vybírá umělý systém cestu **pouze porovnáním počtu kamenů**, tj. je-li ve schránce rozcestníku **více bílých kamenů**, pokračuje **levou** cestou **jinak** pokračuje cestou **pravou**.

Počet cest z uzlu lze jednoduše **rozšířit** přidáním více kamenů **různých barev** (rozdělení počáteční pravděpodobnosti mezi více cest).

Metoda může mít 2 zásadní **problémy**:

- V případě obecného grafu se může **vracet do již dříve vyšetřovaných uzlů** (například v bludišti), může se teoreticky při učení zacyklit.
- Všechny akce provedené na jedné **cestě se hodnotí stejnými vahami**, přestože správná ohodnocení mohou být značně rozdílná (Např. u řešení, které končí v cílovém stavu, je vložen do každé schránky kámen podporující tuto cestu, i když tato cesta nemusí být výhodná - je zbytečně dlouhá atd., lepší by bylo, kdyby odměna/penalizace měli vždy stejně velkou hodnotu, která by byla rozdělena mezi uzly na cestě).

TD learning (On-Policy)

Metoda je založená na **náhodných procházkách**, během kterých ohodnocuje jednotlivé stavy s, **stav s je vždy bezprostředním předchůdcem stavu s'**. Dochází tedy k **šíření hodnot** ze stavů s **odměnou/penalizací**. Šíření ohodnocení je dáno následujícím vzorcem:

ohodnocení(s) = ohodnocení(s) + α *(r(s') + γ *ohodnocení(s') - ohodnocení(s)),

kde:

- **α** : koeficient **učení**,
- **γ** : koeficient určující **vliv** ohodnocení stavu **s'** na ohodnocení předcházejícího stavu **s**,
- **r(s')**: odměna (reward) za dosažení stavu **s'**,

- **ohodnocení(s)**: ohodnocení (utility) stavu **s** při použití dané **strategie** pohybu.

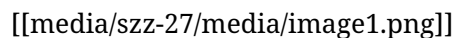
Pro přecházení mezi stavy lze použít různé strategie:

- **Pravděpodobnosti** přechodů mezi sousedními stavy jsou **stejné** a do sousedních stavů přechází **zcela náhodně (random policy)**.
- Pravděpodobnosti jsou **různé** (mohou být například dány **aktuálními hodnotami sousedních stavů**, tj. čím je **vyšší ohodnocení** sousedního stavu, tím je **vyšší pravděpodobnost** přechodu do tohoto stavu).
- Pravděpodobnost jednoho přechodu je **jedničková** a pravděpodobnosti ostatních **nulové**, což je **extrémní případ** předchozího stavu (**greedy policy**).
- **Kombinace předchozích** případů, kdy s pravděpodobnostmi danou parametrem ϵ se použije random policy a s pravděpodobnostmi $(1 - \epsilon)$ se použije **greedy policy** (e-greedy policy).

Princip učení:

1. Zvolte hodnoty koeficientů α a ϵ ($0 < \alpha \leq 1$; $0 < \epsilon \leq 1$) a **vynulujte** ohodnocení všech stavů. Dále **vynulujte počítadlo** procházek **p** a nastavte jejich **maximální** počet na **pmax**. Nastavte **start** $\neq s$.
2. Generujte nový stav **s'** s použitím některé strategie.
3. Přepočítejte novou hodnotu stavu s pomocí vztahu pomocí vzorce výše.
4. Je-li stav **s'** cílovým stavem, pak inkrementuj počet procházek a **start** $\neq s$, jinak **s'** $\neq s$.
5. Je-li $p < pmax$, pak se vraťte na bod 2.

Příklad, vlevo po 1. kroku učení, vpravo po naučení: 



Po naučení se již přechází z libovolného necílového stavu do jeho sousedního stavu, který má **nejvyšší hodnotu** (ale musí být **stejnou nebo lepší** než hodnota aktuálního stavu - to může způsobit **problém uváznutí** (viz modrý stav v obrázku naučeného systému), z nějaké stavu nemusí být dosažitelný žádný jiný stav, lze řešit snížením koeficientu učení α .)

Q learning (Off-Policy)

Metoda je podobná metodě **TD learning**. Tato metoda **místo hodnocení stavů hodnotí akce v těchto stavech (přechody)**. K tomuto hodnocení používá vztah:

$$Q(s, a) = Q(s, a) + \alpha * (r(s') + \epsilon * \max_{a'} Q(s', a') - Q(s, a)),$$

kde:

- α : koeficient **učení**,
- ϵ : koeficient určující **vliv** ohodnocení stavu **s'** na ohodnocení předcházejícího stavu **s**,
- $r(s')$: odměnu (reward) za dosažení stavu **s**,
- $Q(s, a)$: označuje ohodnocení **akce a** provedené **ve stavu s**.
- $\max_{a'} Q(s', a')$: označuje **maximální** hodnotu z **ohodnocení** všech **akcí a'**, které je možné provést ve **stavu s'**.

Metoda je aktivní metodou, tj. bez předem **dané strategie** výběru stavu **s'**. Princip učení:

1. Zvolte hodnoty koeficientů α a ϵ ($0 < \alpha \leq 1$; $0 < \epsilon \leq 1$) a **vynulujte** ohodnocení **Q(s,a)** všech **akcí a** ve všech **stavech s**. Dále vynulujte **počítadlo procházek** $p = 0$ a nastavte jejich maximální počet **pmax**. Nastavte **start** $\neq s$.

2. Vyberte akci **a**, která povede k přechodu ze stavu **s** do stavu **s'**.
3. Vypočítejte novou hodnotu vybrané **akce a** ve **stavu s** pomocí vztahu výše.
4. Je-li **stav s'** cílovým stavem, pak inkrementuj počítadlo procházek a nastav **start** = **s**, jinak nastav **s'** = **s**.
5. Je-li **p < pmax**, pak se **vraťte** na bod 2.

[[media/szz-27/media/image6.png]] Ohodnocení akcí může vypadat následovně: [[media/szz-27/media/image2.png]]

Po naučení se přechází již na základě získaného ohodnocení akcí (v tomto případě přechodů).

SARSA (On-Policy)

Metoda sarsa je přístup, který také ohodnocuje akce, ale k jejich výběru používá nějakou **strategii** π , tzn. **místo** hledání maxima z možných následujících akcí se používá přímo akce $Q(s', a')$ vybraná také **strategií** π . Jde tedy o postup **s, a, r', s', a'** (SARSA). Hlavní funkce pro aktualizaci Q-hodnoty závisí na **aktuálním stavu s**, **akci a**, kterou agent zvolí, **odměně r**, kterou agent **dostane za volbu této akce a**, **stavu s'**, do kterého agent **vstoupí** po provedení **akce a** a nakonec další akci **a'**, kterou agent **zvolí** ve svém **novém stavu**.

Zdroje

- SZZ okruh 27 — studijní materiály FIT BUT (szz-27.docx). Obrázky: media/szz-27/.

Navigace: [[index| • Přehled okruhů]] · [[okruhy/26-reseni-uloh-prohledavani|26. Řešení úloh (prohledávání)]] · další: [[okruhy/28-modelovani-simulace|28. Modelování a simulace]]